Revista Eletrônica de Iniciação Científica em Computação, 2026, XX:1
ISSN: 1519-8219 • doi: 10.5753/reic.2026.XXXXXX
This work is licensed under a Creative Commons Attribution 4.0 International License.

Artigo de Pesquisa

O Custo da Abstração no OpenCV: Análise de Desempenho, Estabilidade e Usabilidade em C++, Java e Python

The Cost of Abstraction in OpenCV: Performance, Stability, and Usability Analysis across C++, Java, and Python

Enzo Rocha L. Diniz Ribas [Centro Universitário Dom Helder Câmara | enzorochaleitedinizribas@gmail.com]
Rafael de Andrade Alves [Centro Universitário Dom Helder Câmara | *rafaaalvescontato@gmail.com*]
Pedro Lucas Vieira De Souza Ramalho [Centro Universitário Dom Helder Câmara |*pedrolucasvieraramalho@gmail.com*]
João Augusto Viera Ramalho [Centro Universitário Dom Helder Câmara | *joaoaugustovr@gmail.com*]
Eduardo do Amaral Melo Pereira [Centro Universitário Dom Helder Câmara | *eduamaralmp@gmail.com*]
Dr. Presleyson Plínio de Lima [Centro Universitário Dom Helder Câmara | *presleyson.lima@academico.domhelder.edu.br*]
Msc. Ricardo Luiz de Freitas [Centro Universitário Dom Helder Câmara | *ricardo.freitas@academico.domhelder.edu.br*]
Published under the Creative Commons Attribution 4.0 International Public License (CC BY 4.0)

Centro Universitário Dom Helder, R. Álvares Maciel, 628 - Santa Efigênia, Belo Horizonte - MG, 30150-250, Brasil.

Resumo. Este artigo investiga o "custo da abstração" no desenvolvimento de aplicações de visão computacional utilizando a biblioteca OpenCV, analisando o impacto da escolha da linguagem de programação no desempenho e na estabilidade do sistema. Por meio de experimentos quantitativos utilizando classificadores *Haar Cascade*, comparou-se a eficiência da implementação nativa em C++ contra ambientes gerenciados em Java e Python. Os resultados demonstram que a abstração impõe um custo mensurável, resultando em um acréscimo de latência entre 13% e 20% nas linguagens interpretadas e gerenciadas, além de maior variabilidade temporal. A análise evidencia que, enquanto C++ oferece o determinismo necessário para sistemas de tempo real e embarcados, Python se destaca pela produtividade em cenários de pesquisa e servidor. Em contrapartida, a implementação em Java apresentou desafios relacionados à imprevisibilidade do *Garbage Collector* e à escassez de suporte da comunidade e ferramentas, tornando-a menos ideal para aplicações críticas de visão. O estudo conclui fornecendo diretrizes para auxiliar na seleção da tecnologia mais adequada, ponderando o *trade-off* entre eficiência computacional e agilidade de desenvolvimento.

Abstract. This paper investigates the "abstraction cost" in developing computer vision applications using the OpenCV library, analyzing the impact of programming language choice on system performance and stability. Through quantitative experiments utilizing *Haar Cascade* classifiers, the efficiency of native C++ implementation was compared against managed environments in Java and Python. The results demonstrate that abstraction imposes a measurable cost, resulting in a latency increase between 13% and 20% in interpreted and managed languages, along with greater temporal variability. The analysis shows that while C++ offers the determinism necessary for real-time and embedded systems, Python stands out for productivity in research and server scenarios. In contrast, the Java implementation presented challenges related to the unpredictability of the *Garbage Collector* and a lack of community support and tools, making it less ideal for critical vision applications. The study concludes by providing guidelines to assist in selecting the most suitable technology, weighing the trade-off between computational efficiency and development agility.

Palavras-chave: Visão Computacional, Linguagens de Programação, Java, C++, Cpp, C, Python, OpenCV

Keywords: Computer Vision, Programming Languages, Java, C++, Cpp, C, Python, OpenCV

Recebido/Received: DD Month YYYY • **Aceito/Accepted:** DD Month YYYY • **Publicado/Published:** DD Month YYYY

1 Introdução

A visão computacional tornou-se um dos pilares fundamentais da tecnologia moderna, impulsionando inovações que variam desde sistemas de condução autônoma e diagnósticos médicos por imagem até autenticação biométrica em dispositivos móveis [Szeliski, 2022]. No centro dessa revolução encontra-se a biblioteca OpenCV (*Open Source Computer Vision Library*), que se estabeleceu como o padrão *de facto* na indústria e na academia devido à sua robustez e vasta coleção de algoritmos otimizados [Bradski, 2000].

Embora o núcleo do OpenCV seja desenvolvido em C++ para garantir máxima eficiência computacional, a biblioteca oferece interfaces (*bindings*) para diversas outras linguagens, como Python e Java. Essa flexibilidade criou um dilema para engenheiros e pesquisadores: optar pela eficiência bruta e controle de memória do C++, ou priorizar a produtividade e a facilidade de desenvolvimento oferecidas por linguagens gerenciadas e de alto nível?

A popularização de linguagens como Python, impulsionada pelo ecossistema de Ciência de Dados, e a onipresença do Java em sistemas corporativos e desenvolvimento Android, levantam questões críticas sobre o "custo da abstração". Até que ponto as camadas adicionais de interpretação, o gerenciamento automático de memória (*Garbage Collection*) e os

mecanismos de interoperação impactam o desempenho em tempo real de algoritmos de visão computacional?

Este trabalho tem como objetivo principal quantificar e analisar esse impacto. Através de um estudo comparativo, avalia-se o desempenho do algoritmo de detecção de placas de carro, utilizando modelos *Haar Cascade*, implementado em C++, Java e Python. A escolha do *Haar Cascade* justifica-se por se tratar de um método amplamente consolidado na literatura, com processamento predominantemente intensivo em CPU e alta sensibilidade a otimizações de baixo nível, características que o tornam particularmente adequado para a análise do custo de abstração imposto por diferentes linguagens de programação e suas respectivas camadas de execução. A análise não se restringe apenas à latência média, mas investiga também qualitativamente a usabilidade, disponibilidade de recursos e a curva de aprendizado de cada ambiente.

As contribuições deste artigo visam fornecer diretrizes empíricas para auxiliar arquitetos de *software* na escolha da pilha tecnológica mais adequada, balanceando os requisitos não-funcionais de desempenho com as necessidades de negócio e produtividade.

Este trabalho está estruturado da seguinte forma: a seção 2 apresenta uma revisão bibliográfica sobre as linguagens de programação e biblioteca estudadas. A seção 3 detalha a metodologia adotada para a implementação e avaliação dos algoritmos. A seção 4 apresenta os resultados obtidos. A seção 5 discute os resultados obtidos, e a seção 6 conclui o artigo, destacando as principais descobertas e sugerindo direções para trabalhos futuros.

2 Revisão Bibliográfica

A revisão bibliográfica permitiu identificar as principais bibliotecas e linguagens de programação utilizadas em visão computacional, bem como suas características, vantagens e desvantagens. A seguir, são apresentadas as principais ferramentas analisadas:

2.1 Visão computacional

Visão computacional é o campo de estudo que busca permitir aos computadores descrever e interpretar o mundo físico. Ela envolve o desenvolvimento de algoritmos e técnicas de processamento, análise e extração de informações de imagens e vídeos. A visão computacional é amplamente utilizada em diversas aplicações, como reconhecimento facial (autenticação visual), detecção de objetos, análise de imagens médicas, modelagem 3D, entre outras [Szeliski, 2022].

2.2 Linguagens de programação

Diversas linguagens de programação são utilizadas para desenvolver aplicações de visão computacional. A seguir, são apresentadas algumas características das principais linguagens analisadas:

- **Java**: Java é uma linguagem de programação que se destaca por ser multiplataforma, segura e orientada a objetos [Schildt, 2019]. Apesar de não ser a principal escolha para visão computacional, possui um conjunto sólido, embora menos extenso, de bibliotecas para visão computacional, como JavaCV, BoofCV e OpenIMAJ, que permitem o desenvolvimento de aplicações nessa área.
- **Python**: Python é uma linguagem de programação amplamente utilizada em áreas da matemática, ciência de dados, inteligência artificial e visão computacional, devido à sua simplicidade, facilidade de aprendizado e utilização, portabilidade e vasta quantidade de bibliotecas [Lutz and Ascher, 1999]. Ela possui um ecossistema robusto para processamento de imagens e visão computacional, com bibliotecas como OpenCV, TensorFlow e MediaPipe, que permitem o desenvolvimento de aplicações complexas de forma acessível e eficiente.
- **C++**: C++ é uma linguagem de programação de alto desempenho utilizada especialmente no desenvolvimento de aplicações que exigem eficiência computacional e controle detalhado dos recursos do sistema. De acordo com Meyers [2014], a linguagem apresenta características de programação de baixo nível permitindo suporte à programação orientada a objetos, possibilitando a criação de aplicações robustas e eficientes. No contexto da visão computacional, C++ é uma das linguagens mais utilizadas, principalmente em aplicações de tempo real, devido à sua alta velocidade de execução. Além disso, algumas das bibliotecas fundamentais para a área foram primeiramente desenvolvidas em C++, como o OpenCV e o Dlib, tornando-a uma escolha consolidada para sistemas de visão computacional de alto desempenho.

2.3 OpenCV

Originalmente lançada por Gary Bradski na Intel em 1999, a OpenCV (*Open Source Computer Vision Library*) foi concebida com o objetivo de acelerar a pesquisa em visão computacional e inteligência artificial, fornecendo uma infraestrutura sólida e acessível para desenvolvedores [Kaehler and Bradski, 2017].

Bradski [2000] define a ferramenta como uma biblioteca de código aberto voltada para a extração e processamento de dados significativos a partir de imagens. A biblioteca é multiplataforma e possui suporte para sistemas operacionais como Linux, Windows, Android e macOS [Kaehler and Bradski, 2017].

Embora sua estrutura principal seja escrita em C e C++ otimizado para eficiência computacional, a OpenCV oferece interfaces ativas para diversas linguagens, incluindo Python, Java e MATLAB [Kaehler and Bradski, 2017]. A literatura aponta que a biblioteca contém mais de 500 funções que abrangem diversas áreas da visão computacional [Kaehler and Bradski, 2017; Bradski, 2000].

Segundo Kaehler and Bradski [2017], essas ferramentas permitem a implementação de tarefas complexas, tais como: inspeção de produtos em fábricas, imagens médicas, segurança, interfaces de usuário, calibração de câmeras, visão estéreo e robótica.

Além disso, a biblioteca é capaz de rastrear movimentos de objetos em 2D ou 3D, determinar formas de objetos a partir de imagens e associar dados de imagem a significados categóricos (como reconhecimento de gestos) [Bradski, 2000]. A ferramenta também integra um módulo completo de aprendizado de máquina (*Machine Learning*), focado em reconhecimento de padrões estatísticos e agrupamento (*clustering*) [Kaehler and Bradski, 2017].

2.4 Trabalhos relacionados

O estudo de Iglovikov [2025] realiza um benchmark abrangente de bibliotecas de decodificação JPEG no ecossistema Python, incluindo OpenCV, Pillow e frameworks de aprendizado de máquina, com foco principal no impacto da implementação interna e do uso de bibliotecas nativas como *libjpeg-turbo* sobre o desempenho. Embora o trabalho discuta explicitamente o custo de camadas de abstração e apresente métricas quantitativas detalhadas, sua análise está restrita à linguagem Python e ao estágio de carregamento e decodificação de imagens, não abordando a comparação entre diferentes linguagens de programação nem a equivalência algorítmica de um mesmo método implementado em múltiplos ambientes.

Já o trabalho de Šrajer *et al.* [2018] investiga o impacto de diferentes ferramentas de diferenciação automática em problemas de visão computacional e aprendizado de máquina, comparando implementações em linguagens como C++, Python, Julia, MATLAB e F#. Apesar de envolver múltiplas linguagens e discutir variações significativas de desempenho associadas a abstrações e estratégias de implementação, o foco do estudo está na computação de derivadas e na eficiência de ferramentas de diferenciação automática, e não na execução de algoritmos clássicos de visão computacional nem no uso direto de bibliotecas amplamente adotadas como o OpenCV em tarefas de detecção de objetos.

Diferentemente desses trabalhos, o presente estudo concentra-se em um algoritmo clássico e amplamente utilizado da biblioteca OpenCV, a detecção de placas veiculares por meio de *Haar Cascade*, permitindo uma análise direta e aplicada do impacto da abstração ao se utilizar a mesma biblioteca em diferentes linguagens de programação, reduzindo vieses introduzidos por diferenças de implementação. Além disso, ao contrário de benchmarks que variam implementações, ferramentas ou níveis de otimização, este trabalho busca manter equivalência algorítmica estrita entre C++, Java e Python, isolando o efeito da linguagem e de suas camadas de abstração sobre desempenho, estabilidade e usabilidade. em cenários práticos de implantação.

3 Metodologia

A metodologia adotada neste trabalho envolve a análise comparativa de diferentes linguagens de programação e a biblioteca OpenCV. O processo metodológico foi dividido nas seguintes etapas:

3.1 Seleção das linguagens e biblioteca

A biblioteca OpenCV foi escolhida devido à sua ampla adoção na comunidade de visão computacional, suporte a múltiplas linguagens e vasta gama de funcionalidades. As linguagens selecionadas para comparação foram C++, Java e Python, considerando sua popularidade, desempenho e facilidade de uso em projetos de visão computacional.

3.2 Definição dos critérios de avaliação

Foram estabelecidos critérios para a avaliação das linguagens e da biblioteca, incluindo desempenho, facilidade de uso e curva de aprendizado, disponibilidade de recursos e comunidade de suporte.

3.2.1 Facilidade de Uso e Curva de Aprendizado

Este critério visa avaliar o quão intuitiva e acessível é a utilização da biblioteca OpenCV em cada linguagem de programação. A avaliação foi realizada através de:

- Análise da documentação oficial: Verificação da clareza, abrangência e exemplos fornecidos na documentação.
- Disponibilidade de tutoriais e recursos online: Pesquisa por tutoriais, cursos e fóruns de discussão relacionados à OpenCV em cada linguagem.
- Configuração do ambiente de desenvolvimento: Avaliação da facilidade de instalação e configuração da biblioteca OpenCV em cada linguagem.
- Experiência prática: Implementação de tarefas básicas de visão computacional para avaliar a curva de aprendizado.
- *Feedback* de desenvolvedores: Coleta de opiniões e experiências de desenvolvedores que utilizaram a OpenCV em cada linguagem.

3.2.2 Desempenho

O desempenho tem o objetivo de medir a eficiência das linguagens e bibliotecas na execução de tarefas comuns em visão computacional, sob cenários controlados. Para avaliar comparativamente o desempenho das linguagens C++, Java e Python em tarefas de Visão Computacional de tempo real.

3.3 Testes Qualitativos

Métrica Qualitativa (Escala Likert): Foi estabelecida uma escala de 1 a 5 (onde 1 = Insuficiente e 5 = Excelente) para classificar três aspectos: (i) clareza da documentação oficial; (ii) disponibilidade de tutoriais/comunidade; e (iii) facilidade de configuração do ambiente (*setup*).

3.4 Testes Quantitativos de Desempenho

Os testes quantitativos foram conduzidos para medir o tempo de execução (latência) de um algoritmo clássico de detecção de objetos em imagens estáticas, utilizando a implementação *Haar Cascade* da biblioteca OpenCV. A escolha do algoritmo se deve à sua eficiência, popularidade e presença em aplicações reais de visão computacional, como reconhecimento facial e detecção de placas veiculares.

Todo o código de teste, *scripts* de medição e procedimentos de pré-processamento foram versionados e disponibilizados no repositório do projeto para garantir reprodutibilidade, [Ribas, 2025].

Para a realização dos testes, foram seguidos os seguintes procedimentos:

3.4.1 Definição do Ambiente de Teste

Ambiente de *Hardware*: Todos os testes foram realizados em uma máquina com as seguintes especificações:

3.4.2 Seleção do Conjuntos de Dados

Para compor o corpus de teste, foram utilizados dois conjuntos de dados públicos de referência (*benchmarks*) obtidos via Kaggle, selecionados pela alta variabilidade de cenários "*in-the-wild*", evitando viés de dados sintéticos ou controlados em laboratório.

Os dados foram organizados em duas fontes para avaliar a robustez dos classificadores:

Tabela 1. Ambiente de Teste e Versões

Linguagem	**Versão**
Python	3.13.5
Java	21
C++ (GCC)	10.3.0
Hardware	
Processador	AMD Ryzen 5 3600X 6 *Cores*
Memória RAM	32 GB
GPU	NVIDIA GeForce RTX 2060
S.O.	Windows 10 Pro 64 bits
Versão do OpenCV	**Método de Instalação**
OpenCV (Python)	4.11 (*bindings* oficiais cv2)
OpenCV (Java/C++)	4.12 (*Local build*)

Dataset A — Diverse LPD (License Plate Detection): Fonte: Diverse-LPD training ready (Mikf, Kaggle). Características: Heterogeneidade geográfica e ambiental, com placas de diversos países (não restritas ao padrão russo dos modelos Haar), ângulos oblíquos, variações de iluminação (sombras, superexposição) e distâncias variadas. Impõe alto desafio de generalização.

Dataset B — License Plates OCR: Fonte: License Plates 1.2M OCR (TrainingDataPro, Kaggle). Características: Conjunto volumoso com foco explícito em veículos; utilizou-se amostragem para validar consistência da detecção em imagens de maior resolução e com foco no objeto de interesse.

3.4.3 Seleção dos Modelos e Parâmetros de Detecção

Os testes foram executados com dois modelos *Haar Cascade* pré-treinados, selecionados para representar diferentes níveis de complexidade computacional:

- `haarcascade_license_plate_rus_16stages.xml`: Modelo com 16 estágios, priorizando velocidade.
- `haarcascade_russian_plate_number.xml`: Modelo focado em detalhes numéricos, com maior densidade de features.

Para garantir a isonomia, o método `detectMultiScale` foi configurado com os mesmos hiperparâmetros nas três implementações:

- **scaleFactor**: 1.1 (redução de 10% na pirâmide de imagem a cada escala);
- **minNeighbors**: 3 (mínimo de vizinhos para validar uma detecção);
- **minSize**: 30×30 *pixels*.

3.4.4 Coleta e Tratamento dos Dados de Desempenho

Para garantir a reprodutibilidade dos testes, foram adotadas múltiplas fontes de consulta, incluindo a documentação oficial das bibliotecas, repositórios de código aberto e artigos técnicos especializados. Nos experimentos práticos, definiram-se dois conjuntos de tarefas-padrão para isolar o desempenho da linguagem:

Os dados brutos de tempo de execução (latência) foram coletados para milhares de iterações. Observou-se que processos em segundo plano do Sistema Operacional e, no caso de linguagens gerenciadas, pausas do *Garbage Collector*, introduziam *outliers* extremos (picos de latência) que não refletiam o desempenho real do algoritmo de visão computacional.

Para mitigar esse ruído e garantir uma comparação justa baseada na eficiência algorítmica, aplicou-se um método robusto de limpeza de dados: Método IQR (Intervalo Interquartil): Foram removidos os registros que excederam o limite superior calculado como $Q3 + 1.5 \times IQR$. Este tratamento eliminou anomalias de sistema (latências > 100ms em processos que levam 5ms), permitindo o uso confiável da Média Aritmética (μ) e do Desvio Padrão (σ) como métricas centrais de avaliação, juntamente com a Mediana para análise de tendências centrais robustas.

3.4.5 Implementação e Coleta de Dados

Para garantir a isonomia do experimento, foram implementados três algoritmos computacionalmente equivalentes em C++, Java e Python. O *pipeline* de processamento obedece à seguinte sequência lógica:

1. **Ingestão da Imagem**: carregamento direto da imagem do disco para a memória utilizando a função `imread` (padrão BGR), sem pré-processamento explícito de cor;
2. **Cronometragem de Precisão**: medição temporal isolada da função `detectMultiScale`, utilizando relógios monotônicos de alta resolução (nanossegundos) para eliminar a latência de I/O de disco;
3. **Pós-processamento**: renderização das *bounding boxes* na imagem original e salvamento em disco para verificação visual;
4. **Persistência**: exportação dos metadados (arquivo, modelo, contagem e tempo em nanossegundos) para arquivos CSV.

Diferentemente de *pipelines* tradicionais, optou-se por passar a matriz de imagem original diretamente ao método de detecção, delegando ao *framework* (OpenCV) qualquer tratamento interno necessário de canais de cor, visando mensurar o tempo total da chamada de API 'como ela é' em uma aplicação final.

O Fluxograma da Figura 1 ilustra o processo completo de cada iteração de teste.

3.4.6 Referências dos Dados

- Créditos para Diverse-LPD training ready de Mikf em Kaggle.com.
- Créditos para License Plates OCR de TrainingDataPro em Kaggle.com.

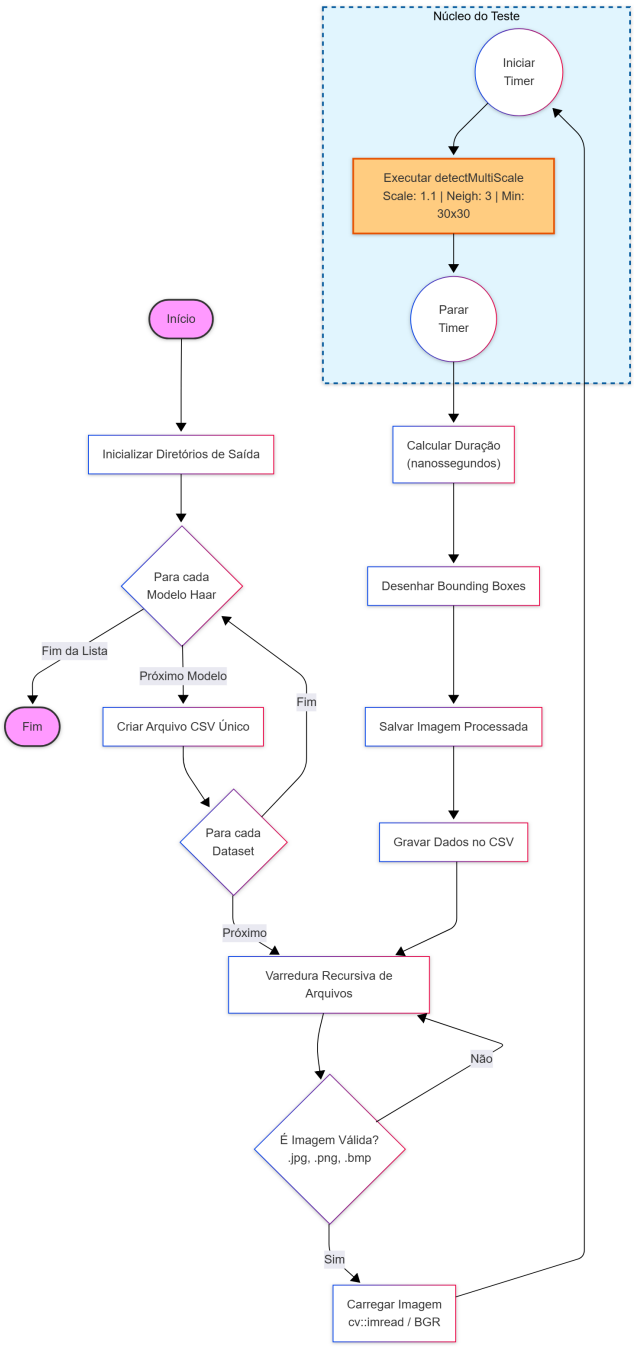


Figura 1. Fluxograma do Pipeline de Teste de Desempenho

4 Resultados

4.1 Resultados Esperados

Com base na metodologia experimental adotada, espera-se que os resultados evidenciem diferenças claras de desempenho, estabilidade e usabilidade entre as linguagens C++, Java e Python ao utilizar a biblioteca OpenCV para tarefas clássicas de visão computacional.

Do ponto de vista de desempenho, é esperado que a implementação em C++ apresente os menores tempos médios de execução e menor variabilidade (desvio padrão), uma vez que a biblioteca é originalmente desenvolvida nessa linguagem e executa sem camadas adicionais de abstração. Espera-se que Python apresente maior latência média, atribuída principalmente à sobrecarga dos *bindings* e à gestão dinâmica de memória, enquanto Java deve ocupar uma posição intermediária, influenciada pelo uso da JVM e pelo comportamento do *Garbage Collector*.

Em relação à estabilidade, espera-se observar maior incidência de *outliers* de latência em linguagens gerenciadas, especialmente em Java, devido a pausas não determinísticas do coletor de lixo. A aplicação do método IQR tende a revelar que, apesar desses picos, a tendência central dos dados permanece consistente, permitindo uma comparação justa entre as linguagens.

No aspecto qualitativo, espera-se que Python seja avaliada como a linguagem com maior facilidade de uso e menor curva de aprendizado, impulsionada por documentação acessível, ampla comunidade e simplicidade sintática. Java deve apresentar boa organização estrutural e robustez, porém com maior complexidade de configuração inicial. C++, embora ofereça máximo controle e desempenho, tende a apresentar maior complexidade sintática e maior esforço de *setup*, refletindo uma curva de aprendizado mais acentuada.

De forma geral, espera-se que os resultados confirmem a existência de um *trade-off* claro entre desempenho bruto e produtividade de desenvolvimento, evidenciando que a escolha da linguagem em projetos de visão computacional deve considerar não apenas a eficiência computacional, mas também fatores como tempo de desenvolvimento, manutenção e perfil da aplicação.

4.2 Resultados Obtidos

4.2.1 Visão Geral dos Dados Coletados

Foram analisadas 100 440 execuções do algoritmo de detecção Haar Cascade, distribuídas igualmente entre as linguagens C++, Java e Python. Cada linguagem processou aproximadamente 33 480 imagens, utilizando dois modelos Haar (16STAGES e RUS).

Antes do tratamento estatístico, os tempos médios globais de execução apresentaram valores de 21,12 ms (C++), 23,82 ms (Java) e 25,49 ms (Python), com ocorrência de picos extremos de latência superiores a 800 ms em todas as linguagens.

A aplicação do método IQR agrupado por linguagem e modelo resultou na remoção de 7 584 registros (7,6% do total), eliminando valores anômalos associados a interferências externas ao algoritmo de visão computacional. Os resultados apresentados a seguir consideram exclusivamente os dados limpos.

4.2.2 Resultados Qualitativos

A Figura 2 apresenta a avaliação qualitativa da facilidade de uso, disponibilidade de recursos e suporte das linguagens C++, Java e Python na implementação do algoritmo Haar Cascade, baseada em um questionário Likert aplicado aos desenvolvedores envolvidos no estudo.

Os resultados qualitativos indicam maior percepção de facilidade de uso para Python, seguido por Java e C++, refletindo a maior abstração e rapidez no desenvolvimento proporcionadas pelas linguagens de mais alto nível. Essa percepção contrasta parcialmente com os resultados quantitativos de desempenho, nos quais C++ apresentou menor latência média e menor variabilidade, evidenciando um *trade-off* entre produtividade percebida e eficiência computacional.

No que se refere à disponibilidade de recursos e suporte, Python e C++ obtiveram avaliações mais elevadas segundo a percepção dos participantes, associadas à ampla comunidade e à abundância de bibliotecas e documentação. A linguagem Java apresentou avaliações inferiores nesse critério no contexto específico do estudo, indicando menor familiaridade ou menor disponibilidade percebida de recursos voltados à visão computacional.

Esses resultados sugerem que, embora C++ ofereça desempenho superior em termos de latência e estabilidade, e Java se posicione de forma intermediária, a escolha da linguagem deve considerar também fatores qualitativos relacionados à produtividade, manutenção, suporte e curva de aprendizado, especialmente em projetos de visão computacional aplicados.

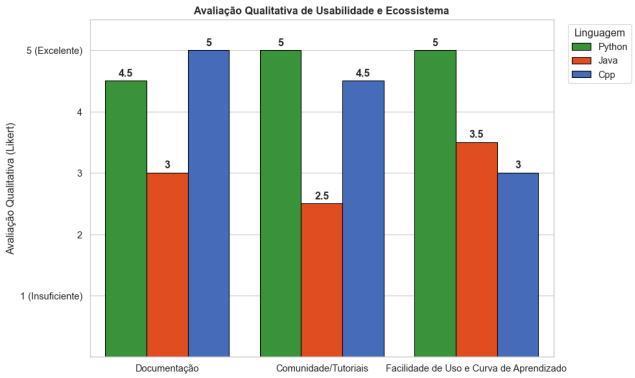


Figura 2. Avaliação qualitativa da facilidade de uso das linguagens C++, Java e Python na implementação do algoritmo Haar Cascade, baseada em questionário Likert aplicado aos desenvolvedores.

4.2.3 Resultados Quantitativos de Desempenho

A Tabela 2 sintetiza os principais resultados quantitativos obtidos após o tratamento estatístico dos dados de latência para cada linguagem e modelo Haar Cascade utilizado.

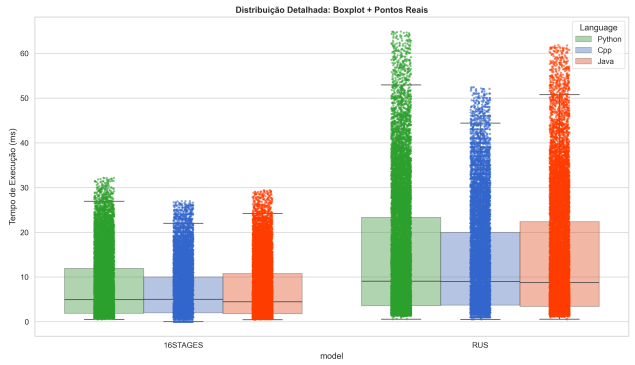


Figura 3. Box plot mostrando a distribuição dos tempos de execução para os modelos 16STAGES e RUS nas linguagens C++, Java e Python.

Em todas as linguagens, aproximadamente 91% das imagens não resultaram em detecções, refletindo a natureza heterogênea do conjunto de imagens junto às limitações inerentes do classificador Haar Cascade quando aplicado a cenários não controlados. A média de detecções por imagem manteve-se estável entre as linguagens, indicando consistência funcional do classificador independentemente da linguagem utilizada.

O gráfico ECDF da Figura 4 ilustra o comportamento da latência acumulada. Observa-se que as três linguagens apresentam tendências similares, com C++ exibindo ligeira vantagem em termos de latência média e menor dispersão na cauda da distribuição.

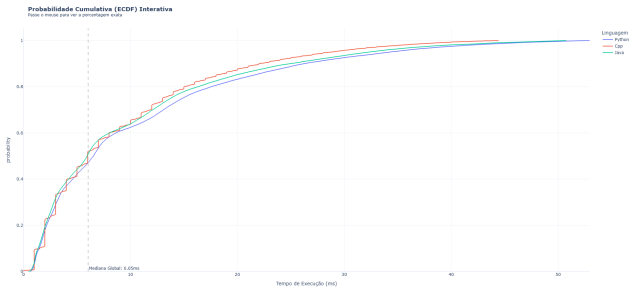


Figura 4. Gráfico ECDF mostrando a latência acumulada em função do número de detecções para os modelos 16STAGES e RUS nas linguagens C++, Java e Python.

A latência média cresce de forma aproximadamente linear à medida que o número de detecções aumenta. Esse comportamento reflete o custo computacional incremental exigido por cada objeto candidato processado, conforme demonstrado nos gráficos de tendência (*OLS Trendline*) da Figura 5. A regressão linear foi empregada com finalidade estritamente exploratória para identificar tendências de crescimento do custo computacional.

A linearidade observada sugere uma relação proporcional entre a densidade de objetos na cena e o tempo de execução, indicando previsibilidade e escalabilidade do sistema dentro do intervalo analisado. Esse padrão é consistente entre C++, Java e Python, embora as diferentes inclinações das retas evidenciem a disparidade na eficiência de execução entre as linguagens compiladas e interpretadas/gerenciadas.

4.2.4 Análise Estatística da Latência

O gráfico de Barras da Figura 6 apresenta a comparação visual das distribuições dos tempos de execução.

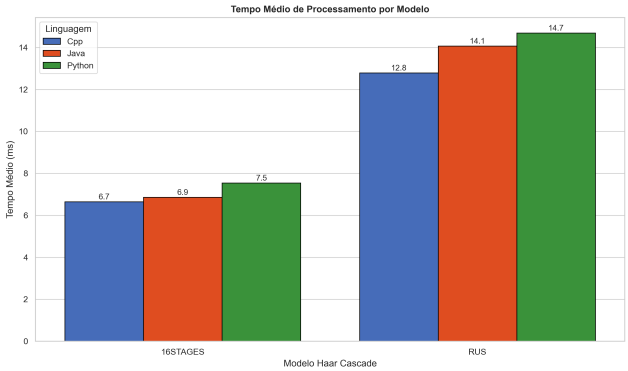
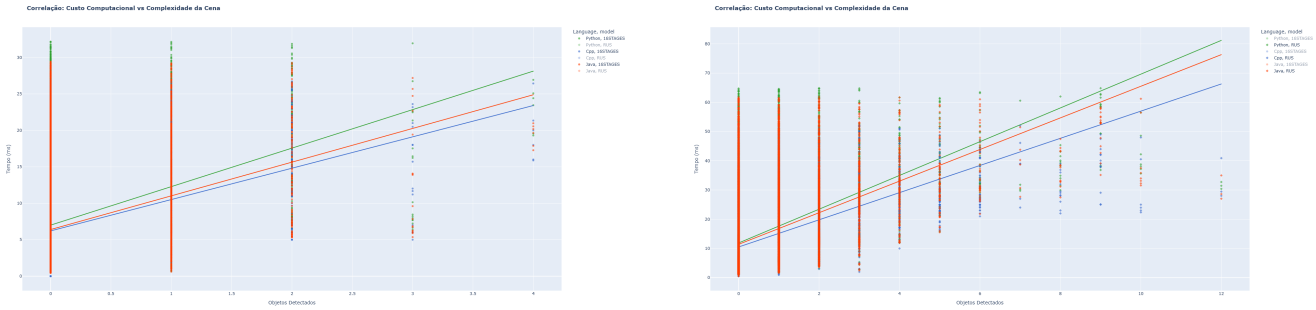


Figura 6. Gráfico de barras comparando os tempos médios de execução entre as linguagens C++, Java e Python.

A concentração dos valores em torno das medianas, evidenciada no *box plot* da Figura 3, sugere que as distribuições, embora apresentem certa assimetria positiva (com médias superiores às medianas devido a valores atípicos de alta latência),

Tabela 2. Resumo estatístico dos experimentos

Linguagem	Modelo	Imagens	Detecções	Vazias (%)	Média (ms)	Mediana (ms)	Desv. Padrão (ms)
C++	16STAGES	15521	1597	91.0	6.65	5.00	5.77
C++	RUS	15488	7589	69.1	12.79	9.00	11.29
Java	16STAGES	15471	1566	91.1	6.86	4.46	6.34
Java	RUS	15467	7612	69.1	14.07	8.75	13.49
Python	16STAGES	15516	1595	91.0	7.54	4.93	6.97
Python	RUS	15393	7492	69.3	14.69	9.07	13.97



Modelo 16STAGES Modelo RUS

Figura 5. Gráficos de regressão linear para os dois modelos comparados.

mantêm-se estáveis. A implementação em C++ apresentou os menores desvios padrão em ambos os modelos, confirmando sua maior estabilidade em comparação aos ambientes gerenciados (Java e Python).

As linguagens Java e Python exibiram maior dispersão, especialmente no modelo RUS, o que sugere maior sensibilidade a variações do ambiente de execução (*runtime overhead*). Após a aplicação do método IQR para limpeza de *outliers*, os valores máximos observados reforçam a hierarquia de desempenho: 26,96 ms para C++, 29,42 ms para Java e 32,17 ms para Python (modelo 16STAGES). No modelo RUS, mais custoso computacionalmente, os máximos atingiram 52,42 ms, 61,85 ms e 64,86 ms, respectivamente.

4.2.5 Síntese Comparativa dos Resultados

De forma geral, os resultados indicam que a implementação em C++ apresentou os menores tempos médios de execução e menor variabilidade em ambos os modelos avaliados. Java e Python apresentaram desempenho intermediário e superior latência média, respectivamente, mantendo, contudo, consistência funcional equivalente no número de detecções realizadas. Os dados evidenciam diferenças mensuráveis de desempenho entre as linguagens analisadas, preservando a equivalência algorítmica e funcional das implementações. A análise qualitativa complementa os achados quantitativos, sugerindo que a escolha da linguagem deve considerar não apenas o desempenho bruto, mas também fatores relacionados à produtividade, facilidade de uso e suporte, especialmente em projetos aplicados.

5 Discussão

Os resultados obtidos confirmam a hipótese inicial de que existe um custo computacional tangível associado à abstração das linguagens de programação ao utilizar a biblioteca OpenCV. A supremacia da implementação em C++ em termos de latência (média de 6,65 ms no modelo 16STAGES) e estabilidade (menor desvio padrão) corrobora a literatura técnica, que atribui essa eficiência à ausência de camadas intermediárias de interpretação e ao gerenciamento manual de memória.

No entanto, a discussão mais relevante reside na análise do trade-off entre eficiência de máquina e eficiência humana. Enquanto Python apresentou a maior latência média (7,54 ms e 14,69 ms para os modelos 16STAGES e RUS, respectivamente), os dados qualitativos da Figura 2 revelam que ela é percebida como a linguagem mais produtiva e acessível pelos desenvolvedores. Isso sugere que, para aplicações em que o tempo de resposta em microssegundos não é crítico (como processamento em batch ou prototipagem acadêmica), como processamento em lote, prototipagem acadêmica ou aplicações exploratórias —, o aumento moderado da latência pode ser compensado por ganhos significativos em velocidade de desenvolvimento e manutenção do software.

O comportamento do Java merece consideração particular. Embora sua média de desempenho seja competitiva, a presença de outliers significativos e a maior dispersão dos dados (especialmente no modelo RUS, com desvio padrão de 13,49 ms) validam a preocupação com o determinismo temporal em ambientes gerenciados. As pausas introduzidas pelo Garbage Collector e a sobrecarga da *Java Virtual Machine* (JVM) tornam essa linguagem menos previsível para sistemas de tempo real rígido (*hard real-time*), conforme antecipado na análise de riscos metodológica. Além disso, a percepção dos desenvolvedores sobre a complexidade do ambiente Java, junto a falta de suporte e ferramentas disponíveis, conforme evidenciado na análise qualitativa, reforça a necessidade de considerar fatores humanos na escolha da linguagem.

A linearidade observada nos gráficos de tendência da Figura 5 demonstra que, apesar das diferenças de velocidade basal, as três linguagens escalam de maneira previsível com o aumento da complexidade da cena.

6 Análises Finais

Este estudo buscou quantificar o impacto da escolha da linguagem de programação no desempenho de algoritmos de visão computacional. As evidências coletadas permitem concluir que o "custo da abstração" é real e mensurável, representando um acréscimo de latência entre 13% e 20% para linguagens gerenciadas (Java e Python) em comparação ao C++ nativo nos cenários testados.

A análise comparativa permite traçar as seguintes diretrizes para a engenharia de *software* em visão computacional:

- **C++:** Permanece a escolha ideal para sistemas embarcados, aplicações críticas de segurança e cenários onde o *throughput* e o determinismo temporal são prioritários, justificando sua curva de aprendizado mais íngreme.
- **Python:** Consolida-se como a melhor ferramenta para pesquisa, desenvolvimento rápido e aplicações em servidor onde a escalabilidade horizontal é viável. A percepção positiva da comunidade e a vastidão do ecossistema compensam a latência adicional em cenários não-críticos.
- **Java:** Apresenta-se como uma opção não ideal para aplicações de visão computacional devido à complexidade do ambiente de execução, falta de suporte e à imprevisibilidade introduzida pelo *garbage collector*. Seu uso deve ser cuidadosamente avaliado frente aos requisitos de desempenho e estabilidade do sistema.

Como trabalhos futuros, sugere-se a extensão da análise para outras bibliotecas populares de visão computacional, como TensorFlow, PyTorch e Keras, bem como a inclusão de métricas adicionais relacionadas ao consumo de memória, eficiência energética e escalabilidade em ambientes distribuídos. Ademais, investigações sobre o impacto de diferentes paradigmas de programação (funcional, orientada a objetos, etc.) no desempenho de algoritmos de visão computacional podem oferecer insights valiosos para a comunidade.

Em suma, não existe uma "melhor linguagem" universal para o OpenCV; a escolha deve ser pautada pelo equilíbrio entre os requisitos não-funcionais de desempenho do sistema e os requisitos de negócio relacionados ao tempo de entrega, manutenibilidade do código e custo de desenvolvimento.

Declarações complementares

Agradecimentos

Agradecimentos a(o):

- Centro Universitário Dom Helder pela infraestrutura disponibilizada para a realização deste trabalho.
- Prof. Dr. Presleyson Lima pela orientação e suporte durante o desenvolvimento deste estudo.
- Prof. Msc. Ricardo Luiz de Freitas pela orientação e valiosas contribuições.
- Prof. Marden Cicarelli pelo auxílio durante a fase de experimentação.
- Demais colegas e colaboradores que contribuíram direta ou indiretamente para a concretização deste trabalho.

Contribuições dos autores

Enzo Rocha Leite Diniz Ribas foi o principal contribuidor e autor deste manuscrito, contribuindo com Conceituação, Metodologia, Análise Formal, Redação, Rascunho Original, Redação, Revisão e Edição e Administração do Projeto.

Ricardo Luiz de Freitas e Presleyson Plínio de Lima contribuíram com administração e supervisão do projeto, supervisão e revisão do manuscrito.

Eduardo do Amaral Melo contribuiu com conceituação, Redação, Rascunho Original, investigação, experimentação, software.

João Augusto Viera Ramalho, Rafael Alves contribuiram com Redação, Rascunho Original, investigação, experimentação e software.

Carlos Eduardo Lopes Caetano, Nicolas Garcia, e Pedro Lucas Vieira Ramalho de Souza contribuíram com investigação, experimentação e software.

Lucas Weiss Telles de Matos contribuiu com conceituação.

Conflitos de interesse

Os autores declaram não haver conflitos de interesse.

Disponibilidade de dados e materiais

Os conjuntos de dados e softwares gerados e analisados durante o estudo atual estão disponíveis em https://github.com/orgs/AtomArm e https://github.com/oEnzoRibas.

Outras informações relevantes

Correspondência deve ser enviada para: Enzo R. L. Diniz Ribas. E-mail: enzorochaleitedinizribas@gmail.com

Referências

Bradski, G. (2000). The opencv library. *Dr. Dobb's Journal*.

Iglovikov, V. (2025). Need for speed: A comprehensive benchmark of jpeg decoders in python.

Kaehler, A. and Bradski, G. (2017). *Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library*. O'Reilly Media, Sebastopol.

Lutz, M. and Ascher, D. (1999). *Learning Python*. O'Reilly Media.

Meyers, S. (2014). *Effective Modern C++*. O'Reilly Media.

Ribas, E. R. L. D. (2025). Github - atom organization page. `https://github.com/AtomArm`. [Online; accessed 14-Jan-2026].

Schildt, H. (2019). *Java: The Complete Reference, Eleventh Edition*. McGraw-Hill Education.

Szeliski, R. (2022). *Computer Vision: Algorithms and Applications 2nd Edition*. Springer.

Šrajer, F., Kukelova, Z., and Fitzgibbon, A. (2018). A benchmark of selected algorithmic differentiation tools on some problems in computer vision and machine learning.